

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)

КБ-2 «Информационно-аналитические системы кибербезопасности»

ОТЧЕТ О ВЫПОЛНЕНИИ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ №6

В РАМКАХ ДИСЦИПЛИНЫ «РАЗРАБОТКА ПРОГРАМНЫХ СРЕДСТВ ПОИСКА И АНАЛИЗА КИБЕРУГРОЗ»

Выполнил:

Студент 5-ого курса

Учебной группы БИСО-02-22

Зубарев В.С.

Москва 2026

Практическая работа № 6

Кодирование категориальных признаков в кибербезопасности

Решение выполнено на приложенном `soc_categorical_encoding_lab.csv`. Ячейки запускаются сверху вниз. Все словари, категории кодировщиков и статистики целевого кодирования получены только из `train`; для `test` используется только применение готовых правил. Выводы находятся в текстовых ячейках.

Приложенный PDF содержит титульный лист отчёта без содержания лекции, поэтому методы и требования сверены с рабочей тетрадью задания.

0. Подготовка данных и вспомогательные функции

Сохраняем исходное разделение `train` / `test`. Время нужно для упорядоченного целевого кодирования, а `is_attack` служит целевой меткой и не является входным признаком.

In [3]:
```
from pathlib import Path
import hashlib

import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 150)

# Ищем приложенный CSV в той же папке, где запускается блокнот.
data_path = Path('soc_categorical_encoding_lab.csv')
if not data_path.exists():
    raise FileNotFoundError('Положите soc_categorical_encoding_lab.csv рядом с б

df = pd.read_csv(data_path, parse_dates=['event_time'])
train = df[df['dataset_part'] == 'train'].copy()
test = df[df['dataset_part'] == 'test'].copy()

print('Всего:', df.shape, '| train:', len(train), '| test:', len(test))
display(df.head())
```

```
Всего: (1600, 14) | train: 1200 | test: 400
```

	event_id	event_time	dataset_part	protocol	direction	service	country	alert_rule	
0	EVT-000001	2026-08-01 00:02:43	train	TCP	outbound	dns	NL	RULE-001	1
1	EVT-000002	2026-08-01 00:03:56	train	ICMP	outbound	rdp	US	RULE-028	
2	EVT-000003	2026-08-01 00:04:21	train	ICMP	outbound	ssh	CN	RULE-001	1
3	EVT-000004	2026-08-01 00:04:56	train	TCP	inbound	smb	DE	RULE-013	
4	EVT-000005	2026-08-01 00:07:37	train	UDP	outbound	ssh	NL	RULE-001	1

In [4]:
```python
def stable_bucket(value, number_of_buckets):
    # SHA-256 всегда даёт одинаковую корзину для одной и той же строки.
    digest = hashlib.sha256(str(value).encode('utf-8')).hexdigest()
    return int(digest, 16) % number_of_buckets


def smoothed_target_stats(data, feature, m=20):
    # Общая доля атак и статистика каждой категории берутся только из train.
    prior = data['is_attack'].mean()
    statistics = data.groupby(feature)['is_attack'].agg(['sum', 'count'])
    statistics['encoded'] = (
        statistics['sum'] + m * prior
    ) / (statistics['count'] + m)
    return prior, statistics
```

Задание 1. Аудит признаков и выбор методов

Номинальные признаки обозначают названия без естественного порядка. Порядковые признаки имеют заданную последовательность уровней. Кардинальность считаем по `train`, так как именно на нём обучаются кодировщики.

In [6]:
```python
nominal_cols = [
    'protocol', 'direction', 'service', 'country',
    'alert_rule', 'src_ip', 'domain'
]
ordinal_cols = ['severity', 'confidence', 'asset_criticality']

print('Размер:', df.shape)
print('Типы столбцов:')
display(df.dtypes.to_frame('Тип'))
print('Пропусков:', int(df.isna().sum().sum()))
print('Полных дубликатов:', int(df.duplicated().sum()))
print('Номинальные:', ', '.join(nominal_cols))
print('Порядковые:', ', '.join(ordinal_cols))

# Число категорий в test показываем лишь для проверки новых значений.
cardinality = pd.DataFrame({
```

```
    'Категорий в train': train[nominal_cols + ordinal_cols].nunique(),
    'Категорий в test': test[nominal_cols + ordinal_cols].nunique()
})
display(cardinality)

method_choice = pd.DataFrame({
    'Признак': ['protocol', 'service', 'alert_rule', 'src_ip', 'country', 'sever
    'Категорий в train': [train[column].nunique() for column in
                          ['protocol', 'service', 'alert_rule', 'src_ip', 'coun
    'Подход': [
        'One-Hot; Label только для сравнения',
        'One-Hot или Binary при экономии столбцов',
        'Hashing с контролем коллизий',
        'Hashing или признаки подсети/редкости',
        'One-Hot или сглаженный Target Encoding',
        'Ordinal или Thermometer с явным порядком'
    ],
    'Причина': [
        'Протоколы не имеют числового порядка.',
        'Категорий немного; бинарный код компактнее.',
        'Много правил; новые правила возможны в test.',
        'Много адресов и новые адреса в test.',
        'Новые страны требуют безопасного значения по умолчанию.',
        'Уровни имеют заданную последовательность.'
    ]
})
display(method_choice)
```

```
Размер: (1600, 14)
Типы столбцов:
Пропусков: 0
Полных дубликатов: 0
Номинальные: protocol, direction, service, country, alert_rule, src_ip, domain
Порядковые: severity, confidence, asset_criticality
```

	Тип
event_id	str
event_time	datetime64[us]
dataset_part	str
protocol	str
direction	str
service	str
country	str
alert_rule	str
src_ip	str
domain	str
severity	str
confidence	str
asset_criticality	str
is_attack	int64

	Категорий в train	Категорий в test
protocol	3	4
direction	2	2
service	7	8
country	10	11
alert_rule	36	41
src_ip	207	183
domain	145	156
severity	4	4
confidence	3	3
asset_criticality	4	4

	Признак	Категорий в train	Подход	Причина
0	protocol	3	One-Hot; Label только для сравнения	Протоколы не имеют числового порядка.
1	service	7	One-Hot или Binary при экономии столбцов	Категорий немного; бинарный код компактнее.
2	alert_rule	36	Hashing с контролем коллизий	Много правил; новые правила возможны в test.
3	src_ip	207	Hashing или признаки подсети/редкости	Много адресов и новые адреса в test.
4	country	10	One-Hot или сглаженный Target Encoding	Новые страны требуют безопасного значения по умолчанию.
5	severity	4	Ordinal или Thermometer с явным порядком	Уровни имеют заданную последовательность.

Вывод по заданию 1

В наборе 1600 строк: 1200 обучающих и 400 тестовых, без пропусков и полных дубликатов. В `train` есть 3 протокола, 7 сервисов, 36 правил оповещения, 207 адресов `src_ip`, 10 стран и 4 уровня `severity`. Идентификаторы `event_id` и `src_ip` нельзя считать непрерывными числами; `event_id` вообще не нужен как входной признак. Высокая кардинальность `src_ip` и `domain` делает полный One-Hot громоздким. Для `severity`, `confidence` и `asset_criticality` порядок необходимо записывать явно.

Задание 2. Label, One-Hot и Dummy Encoding

`OrdinalEncoder` здесь используется как аналог Label Encoding для номинального протокола. Его числа не означают степень опасности. One-Hot выделяет по одному столбцу на известную категорию. Dummy Encoding удаляет один базовый столбец, что сокращает размерность, но меняет представление неизвестных категорий.

In [9]:
```
# OrdinalEncoder выучит только три категории из train.
label_encoder = OrdinalEncoder(
    handle_unknown='use_encoded_value', unknown_value=-1
)
protocol_train_label = label_encoder.fit_transform(train[['protocol']])
protocol_test_label = label_encoder.transform(test[['protocol']])

# OneHotEncoder для неизвестного протокола выдаст строку из нулей.
onehot_encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
protocol_train_onehot = onehot_encoder.fit_transform(train[['protocol']])
protocol_test_onehot = onehot_encoder.transform(test[['protocol']])
onehot_names = onehot_encoder.get_feature_names_out(['protocol'])

# Базовая категория для Dummy выбирается в train. Для test строим все
```

```
# индикаторы и оставляем только обучающие столбцы.
protocol_train_dummy = pd.get_dummies(train['protocol'], drop_first=True, dtype=
protocol_test_all = pd.get_dummies(test['protocol'], dtype=int)
protocol_test_dummy = protocol_test_all.reindex(
    columns=protocol_train_dummy.columns, fill_value=0
)

quic_rows = test['protocol'] == 'QUIC'
print('Категории, известные Label Encoder:', label_encoder.categories_[0])
print('Столбцы: Label = 1, One-Hot =', protocol_train_onehot.shape[1],
      ', Dummy =', protocol_train_dummy.shape[1])
print('Событий QUIC в test:', int(quic_rows.sum()))

quic_example = pd.DataFrame({
    'protocol': test.loc[quic_rows, 'protocol'].to_numpy(),
    'Label': protocol_test_label[quic_rows.to_numpy(), 0],
    'Сумма One-Hot': protocol_test_onehot[quic_rows.to_numpy()].sum(axis=1),
    'Сумма Dummy': protocol_test_dummy.loc[quic_rows].sum(axis=1).to_numpy()
})
display(quic_example.head())
```

```
Категории, известные Label Encoder: ['ICMP' 'TCP' 'UDP']
Столбцы: Label = 1, One-Hot = 3 , Dummy = 2
Событий QUIC в test: 18
```

	protocol	Label	Сумма One-Hot	Сумма Dummy
0	QUIC	-1.0	0.0	0
1	QUIC	-1.0	0.0	0
2	QUIC	-1.0	0.0	0
3	QUIC	-1.0	0.0	0
4	QUIC	-1.0	0.0	0

Вывод по заданию 2

В `train` кодировщики увидели ICMP, TCP и UDP, поэтому Label Encoding даёт 1 столбец, One-Hot — 3, Dummy — 2. Для 18 событий `QUIC` в `test` Label Encoding возвращает −1, а One-Hot — все нули. Dummy тоже даёт все нули, но такую строку невозможно отличить от базовой известной категории ICMP, столбец которой был удалён. Номинальные числа Label Encoding могут навязать модели несуществующее расстояние между протоколами; для большинства моделей безопаснее One-Hot. Отдельная категория «unknown» нужна, если неизвестные протоколы должны различаться с базовым уровнем.

Задание 3. Binary, Count/Frequency и Feature Hashing

Для Binary Encoding код 0 резервируем для неизвестного `service`, а известным категориям даём номера с 1. Поэтому нужен минимум битов, достаточный и для нуля, и для всех известных сервисов. Count/Frequency кодируют

распространённость в `train`, но не различают категории с одинаковой частотой. Хеширование ограничивает число столбцов ценой коллизий.

In [12]:
```
service_names = sorted(train['service'].unique())
service_codes = {name: number for number, name in enumerate(service_names, start
number_of_bits = len(service_names).bit_length()

def service_bits(value):
    # Неизвестный сервис получает код 0: все биты равны нулю.
    number = service_codes.get(value, 0)
    binary_text = format(number, f'0{number_of_bits}b')
    return [int(digit) for digit in binary_text]

bit_columns = [f'service_bit_{i + 1}' for i in range(number_of_bits)]
service_train_binary = pd.DataFrame(
    [service_bits(value) for value in train['service']], columns=bit_columns,
    index=train.index
)
service_test_binary = pd.DataFrame(
    [service_bits(value) for value in test['service']], columns=bit_columns,
    index=test.index
)

# Частоты известны только по обучающим строкам.
service_counts = train['service'].value_counts()
service_train_count = train['service'].map(service_counts).fillna(0)
service_test_count = test['service'].map(service_counts).fillna(0)
service_train_frequency = service_train_count / len(train)
service_test_frequency = service_test_count / len(train)

service_summary = pd.DataFrame({
    'Код': pd.Series(service_codes),
    'Count': service_counts,
    'Frequency': service_counts / len(train)
}).sort_index()
display(service_summary.round(3))
print('Известных сервисов:', len(service_names), '| двоичных столбцов:', number_
print('mqtt в test:', int((test['service'] == 'mqtt').sum()))
display(service_test_binary.loc[test['service'] == 'mqtt'].head())
print('Count и Frequency для mqtt:',
      service_test_count[test['service'] == 'mqtt'].unique(),
      service_test_frequency[test['service'] == 'mqtt'].unique())
```

```
Известных сервисов: 7 | двоичных столбцов: 3
mqtt в test: 16
Count и Frequency для mqtt: [0.] [0.]
```

	Код	Count	Frequency
dhcp	1	89	0.074
dns	2	198	0.165
https	3	402	0.335
none	4	112	0.093
rdp	5	101	0.084
smb	6	110	0.092
ssh	7	188	0.157

	service_bit_1	service_bit_2	service_bit_3
1213	0	0	0
1226	0	0	0
1230	0	0	0
1245	0	0	0
1269	0	0	0

In [13]:
```
number_of_buckets = 8

# Каждое правило всегда попадает в одну из восьми корзин.
train_rule_buckets = train['alert_rule'].map(
    lambda value: stable_bucket(value, number_of_buckets)
)
test_rule_buckets = test['alert_rule'].map(
    lambda value: stable_bucket(value, number_of_buckets)
)

# Восемь двоичных столбцов — собственно хешированные признаки.
hashed_train = pd.get_dummies(train_rule_buckets, dtype=int).reindex(
    columns=range(number_of_buckets), fill_value=0
)
hashed_test = pd.get_dummies(test_rule_buckets, dtype=int).reindex(
    columns=range(number_of_buckets), fill_value=0
)
hashed_train.columns = [f'rule_hash_{i}' for i in range(number_of_buckets)]
hashed_test.columns = hashed_train.columns

rule_to_bucket = pd.DataFrame({
    'alert_rule': sorted(train['alert_rule'].unique())
})
rule_to_bucket['Корзина'] = rule_to_bucket['alert_rule'].map(
    lambda value: stable_bucket(value, number_of_buckets)
)
used_buckets = rule_to_bucket['Корзина'].nunique()
extra_collisions = len(rule_to_bucket) - used_buckets
collision_buckets = int((rule_to_bucket['Корзина'].value_counts() > 1).sum())

print('Правил в train:', len(rule_to_bucket))
print('Использовано корзин:', used_buckets, 'из', number_of_buckets)
print('Правил сверх первого в своих корзинах:', extra_collisions)
```

```
print('Корзин с коллизиями:', collision_buckets)
display(rule_to_bucket.head(10))

new_rules = sorted(set(test['alert_rule']) - set(train['alert_rule']))
new_rule_buckets = pd.DataFrame({'Новое правило': new_rules})
new_rule_buckets['Корзина'] = new_rule_buckets['Новое правило'].map(
    lambda value: stable_bucket(value, number_of_buckets)
)
print('Новых правил в test:', len(new_rules))
display(new_rule_buckets)
```

```
Правил в train: 36
Использовано корзин: 8 из 8
Правил сверх первого в своих корзинах: 28
Корзин с коллизиями: 7
Новых правил в test: 5
```

	alert_rule	Корзина
0	RULE-001	6
1	RULE-002	4
2	RULE-003	6
3	RULE-004	0
4	RULE-005	0
5	RULE-006	7
6	RULE-007	4
7	RULE-008	0
8	RULE-009	2
9	RULE-010	0

	Новое правило	Корзина
0	RULE-101	3
1	RULE-102	6
2	RULE-103	4
3	RULE-104	5
4	RULE-105	1

Вывод по заданию 3

Для 7 известных сервисов Binary Encoding с резервным нулём требует 3 столбца, тогда как One-Hot потребовал бы 7. Все 16 событий `mqtt` в `test` получают двоичный код 000 и Count/Frequency 0, поскольку сервис не встречался в `train`. Count и Frequency дают один понятный столбец, однако одинаково частые сервисы получают одинаковое число; отдельные двоичные биты менее понятны SOC. Все 36 обучающих `alert_rule` распределились по 8 хеш-корзинам: 28 правил оказались сверх первого в своих корзинах, а 7 корзин содержат коллизии. Хеширование

сохраняет фиксированную размерность и принимает новые правила, но восемь корзин дают существенную потерю различимости; для реальной системы число корзин следовало бы увеличить и проверить качество модели.

Задание 4. Target, Leave-One-Out и Ordered Target Encoding

Целевое кодирование использует `is_attack`, поэтому особенно важно исключать утечку метки. Сглаживание с `m = 20` притягивает оценки редких стран к общей доле атак в `train`. Для LOO обучающая строка исключается из статистики своей категории. Ordered использует только более ранние обучающие события.

In [16]:
```
# Target Encoding для country. Неизвестная страна получает общий prior.
country_prior, country_stats = smoothed_target_stats(train, 'country', m=20)
country_train_target = train['country'].map(country_stats['encoded'])
country_test_target = test['country'].map(country_stats['encoded']).fillna(count

print('Доля атак в train (prior):', round(country_prior, 4))
display(country_stats.round(4))
print('Неизвестных стран BR в test:', int((test['country'] == 'BR').sum()))
print('Их код:', country_test_target[test['country'] == 'BR'].unique())

# LOO для protocol: вычитаем метку текущей строки из суммы категории.
protocol_sum = train.groupby('protocol')['is_attack'].transform('sum')
protocol_count = train.groupby('protocol')['is_attack'].transform('count')
protocol_train_loo = (protocol_sum - train['is_attack']) / (protocol_count - 1)
protocol_train_loo = protocol_train_loo.fillna(country_prior)

# Для test допустимо использовать все метки train, но ни одну метку test.
protocol_train_mean = train.groupby('protocol')['is_attack'].mean()
protocol_test_loo = test['protocol'].map(protocol_train_mean).fillna(country_pri
assert train['event_time'].max() < test['event_time'].min()

# Сортировка делает правило «только предыдущие события» явным.
ordered_train = train.sort_values(['event_time', 'event_id']).copy()
ordered_values = []
category_sums = {}
category_counts = {}

for _, row in ordered_train.iterrows():
    category = row['protocol']
    if category_counts.get(category, 0) > 0:
        value = category_sums[category] / category_counts[category]
    else:
        value = 0.5  # До первого события этой категории нет её прошлых меток.
    ordered_values.append(value)

    # Текущую метку добавляем только после расчёта её кода.
    category_sums[category] = category_sums.get(category, 0) + row['is_attack']
    category_counts[category] = category_counts.get(category, 0) + 1

ordered_train['protocol_ordered'] = ordered_values
protocol_train_ordered = ordered_train['protocol_ordered'].reindex(train.index)
protocol_test_ordered = test['protocol'].map(protocol_train_mean).fillna(country

comparison = pd.DataFrame({
```

```
    'protocol': train['protocol'],
    'is_attack': train['is_attack'],
    'LOO': protocol_train_loo,
    'Ordered': protocol_train_ordered
})
display(comparison.head(10).round(3))
print('Первая по времени обучающая строка:')
display(ordered_train[['event_time', 'protocol', 'is_attack', 'protocol_ordered'
print('Test начинается после train: да')
```

```
Доля атак в train (prior): 0.4392
Неизвестных стран BR в test: 14
Их код: [0.43916667]
Первая по времени обучающая строка:
Test начинается после train: да
```

country	sum	count	encoded
CN	77	145	0.5199
DE	66	178	0.3777
FR	24	61	0.4047
GB	28	67	0.4228
IN	37	88	0.4239
IR	47	94	0.4893
KP	11	19	0.5073
NL	43	107	0.4077
RU	89	204	0.4365
US	105	237	0.4427

	protocol	is_attack	LOO	Ordered
0	TCP	1	0.425	0.500
1	ICMP	1	0.449	0.500
2	ICMP	0	0.459	1.000
3	TCP	0	0.426	1.000
4	UDP	1	0.475	0.500
5	TCP	0	0.426	0.500
6	TCP	1	0.425	0.333
7	TCP	0	0.426	0.500
8	TCP	0	0.426	0.400
9	TCP	1	0.425	0.333

	event_time	protocol	is_attack	protocol_ordered
0	2026-08-01 00:02:43	TCP	1	0.5
1	2026-08-01 00:03:56	ICMP	1	0.5
2	2026-08-01 00:04:21	ICMP	0	1.0
3	2026-08-01 00:04:56	TCP	0	1.0
4	2026-08-01 00:07:37	UDP	1	0.5

Вывод по заданию 4

Общая доля атак в `train` равна 0,4392; это код для 14 тестовых событий из неизвестной страны `BR`. Сглаженный Target Encoding уменьшает шум редких стран, но код обучающей страны всё ещё учитывает метку текущей строки, поэтому сглаживание само по себе не устраняет утечку при обучении модели. LOO исключает текущую метку, но по-прежнему использует метки других, в том числе более поздних, строк. Ordered для обучающей строки использует только предыдущие события того же протокола; до первой встречи протокола применяется фиксированный код 0,5. `Test` начинается позже `train`, поэтому при его кодировании допустимы все метки обучающей части и недопустимы метки самого теста.

Задание 5. Порядковые признаки

Для `severity` задаём явный порядок `low < medium < high < critical`. Ordinal Encoding даёт числа 0–3 и может навязать равные расстояния между уровнями. One-Hot не предполагает расстояния и порядок. Thermometer показывает пересечение трёх порогов и сохраняет структуру порядка.

In [19]:
```
severity_order = ['low', 'medium', 'high', 'critical']

severity_ordinal_encoder = OrdinalEncoder(
    categories=[severity_order],
    handle_unknown='use_encoded_value', unknown_value=-1
)
severity_train_ordinal = severity_ordinal_encoder.fit_transform(
    train[['severity']]
).ravel()
severity_test_ordinal = severity_ordinal_encoder.transform(
    test[['severity']]
).ravel()

severity_onehot_encoder = OneHotEncoder(
    categories=[severity_order], handle_unknown='ignore', sparse_output=False
)
severity_train_onehot = severity_onehot_encoder.fit_transform(train[['severity']
severity_test_onehot = severity_onehot_encoder.transform(test[['severity']])

# Три столбца показывают, достигнут ли каждый следующий порог.
severity_train_thermometer = pd.DataFrame({
```

```
    'at_least_medium': (severity_train_ordinal >= 1).astype(int),
    'at_least_high': (severity_train_ordinal >= 2).astype(int),
    'at_least_critical': (severity_train_ordinal >= 3).astype(int)
}, index=train.index)
severity_test_thermometer = pd.DataFrame({
    'at_least_medium': (severity_test_ordinal >= 1).astype(int),
    'at_least_high': (severity_test_ordinal >= 2).astype(int),
    'at_least_critical': (severity_test_ordinal >= 3).astype(int)
}, index=test.index)

severity_examples = pd.DataFrame({'severity': severity_order})
severity_examples['Ordinal'] = range(len(severity_order))
for threshold, column in [(1, 'at_least_medium'), (2, 'at_least_high'),
                          (3, 'at_least_critical')]:
    severity_examples[column] = (severity_examples['Ordinal'] >= threshold).asty
display(severity_examples)
print('Столбцов: Ordinal = 1, One-Hot =', severity_train_onehot.shape[1],
      ', Thermometer =', severity_train_thermometer.shape[1])

severity_prior, severity_stats = smoothed_target_stats(train, 'severity', m=20)
severity_stats = severity_stats.reindex(severity_order)
severity_train_target = train['severity'].map(severity_stats['encoded'])
severity_test_target = test['severity'].map(severity_stats['encoded']).fillna(se
display(severity_stats.round(4))
```

```
Столбцов: Ordinal = 1, One-Hot = 4 , Thermometer = 3
```

	severity	Ordinal	at_least_medium	at_least_high	at_least_critical
0	low	0	0	0	0
1	medium	1	1	0	0
2	high	2	1	1	0
3	critical	3	1	1	1

severity	sum	count	encoded
low	153	511	0.3047
medium	163	331	0.4894
high	121	259	0.4652
critical	90	99	0.8301

Вывод по заданию 5

Ordinal сохраняет порядок уровней, но его числа предполагают одинаковый шаг от `low` к `medium` и от `high` к `critical`. One-Hot не делает этого предположения, однако теряет видимую связь соседних уровней. Thermometer хранит последовательность порогов в трёх столбцах; модель может оценивать каждый переход отдельно. Сглаженные доли атак равны 0,3047 (`low`), 0,4894 (`medium`), 0,4652 (`high`) и 0,8301 (`critical`): `medium` здесь выше `high` , хотя порядок серьёзности обратный. Эти доли — статистика `train` , а не определение уровней.

Для обучения модели целевые коды самих обучающих строк нужно получать без их текущих меток, например LOO или Ordered.